Intro To ML
Prof. Mathieu Salzmann

Luigi Ieracitano

March 12, 2026

Contents

1 Introduction **2**

1.1 What is Machine Learning ? . . . 2

1.1.1 Data . . . 2

1.1.2 Insight (what we discover from the datas) . . . 2

1.1.3 Algorithms . . . 3

2 Supervised Linear Model **4**

2.1 Notation . . . 4

2.2 Linear Regression . . . 4

2.2.1 Training . . . 4

2.2.2 Prediction . . . 5

2.2.3 1D linear regression . . . 5

2.2.4 Two input dimensions . . . 6

2.2.5 Hyperplane . . . 6

2.2.6 Minimizing multivariate function . . . 6

2.2.7 Solution of the linear regression . . . 6

2.2.8 Test time . . . 8

2.3 Model evaluation . . . 8

2.3.1 Evaluation metrics for regression . . . 8

2.3.2 Linear regression with multiple output dimensions . . . 9

2.3.3 Model . . . 9

2.4 Linear classification . . . 10

2.4.1 Logistic regression . . . 11

2.4.2 Difference between linear models . . . 13

2.4.3 Support Vector Machine (SVM) . . . 13

2.4.4 Classification evaluation . . . 14

2.4.5 From binary to multi-class classification . . . 15

2.4.6 Decision boundaries: General Definition . . . 15

2.4.7 Classification evaluation for Multi-class . . . 16

2.4.8 Least-square classification . . . 16

2.4.9 Multi-class logistic regression . . . 18

2.4.10 Dealing with multiple classes . . . 19

2.4.11 Comparing linear classifiers . . . 19

2.4.12 Recap - and new challenge . . . 20

Introduction

1.1 What is Machine Learning ?

Machine learning = Data →Algorithms →Insight

1.1.1 Data

Data = attributes/tabular, text, speech, images, mixed.

Definition 1 (Data sample)**.** An individual observation. E.g., an image, or a list of attributes for one patient

Definition 2 (Data set)**.** A collection of multiple data sample. E.g., a collection of images, the attribute lists for multiple patients.

Unsupervised data : Each sample consists only of an observation, e.g., an image (without information about its content).
Supervised data : Each sample comes with additional annotations, e.g., category label.

1.1.2 Insight (what we discover from the datas)

What insight ?

- Precise, concrete prediction. E.g., the category depicted by an image.
- Better understanding of a phenomenon. E.g., identify the mother's charasteristics that lead to low birth weight.
- Better understanding of data. E.g., analyze the influences between different musical genres.
- **in this course** we mostly focus on precise insights. In particular we will study two classes of ML problems : **regression** and **classification**

Regression : Predict continuous value(s) for a given sample. E.g., predict the birth weight of a baby, human pose estimation : predict the 3D positions of human joints (multiple values).
Classification : Predict one discrete label for a given sample. E.g., binary classifications for movies (like vs not like; 1 or 0), muti-class image recognition (cat vs tree vs car; 0 vs 1 vs 2).

Order:
In regression, the values typically follow an order. E.g, predicting a weight of 3002g insted of 2977g is better that predicting 2500g.
In classification, the categories do not follow an order. E.g, predicting a "tree" instead of a "cat" is just as wrong as predicting "car". →here it's true or false.

1.1.3 Algorithms

What (classes of) algorithms ?

- Supervised learning :
  - Relies on supervised data
  - The annotations typically correspond to the desired insight
- Unsupervised learning :
  - Relies on unsupervised data
  - The goal is rather to analyze the observed data set

Unsupervised learning:

- Single stage: Analyze the whole data or transform it for further analysis.

Supervised learning:

- Stage 1: Training: Use data with ground-truth (info that is true) labels to optimize model parameters.
- Stage 2: Testing: Predict the output for a new data sample.
- Assumption : The annotations (correct responses) are the insight that we seek to obtain from the data. E.g.,: category label, human pose

Training set vs test set:

- The training set and the test set should always be **completely separate**.
- Never use the test annotations during training →the test set is not to train or to improve the performance of the model, but only to test his efficiency.
- Assumption: they are both drawn from the same statistical distribution. E.g., digit recognition: A model trained on MNIST will work poorly on Street View House Numbers

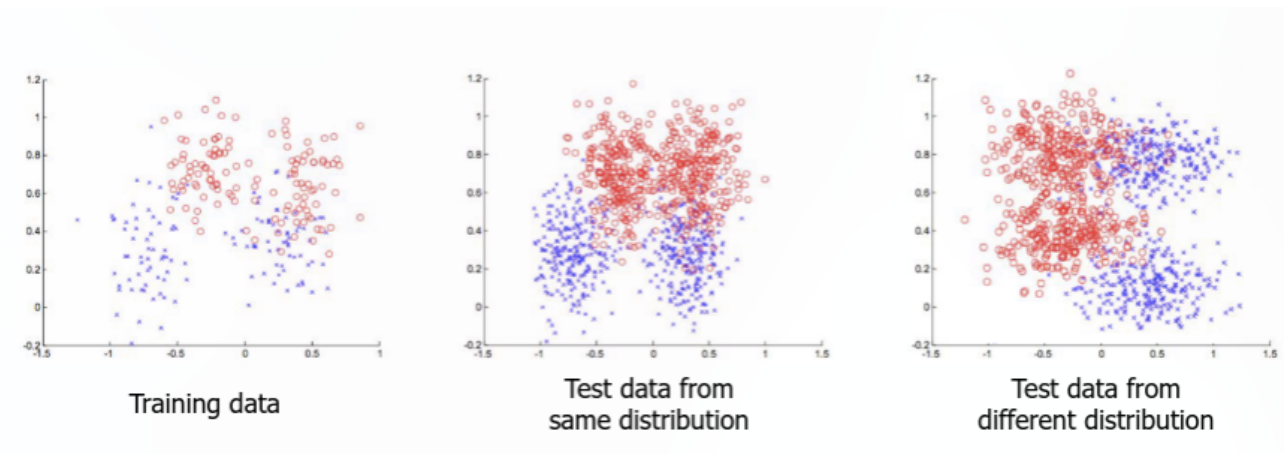


Figure 1.1: Same distribution assumption

Supervised Linear Model

2.1 Notation

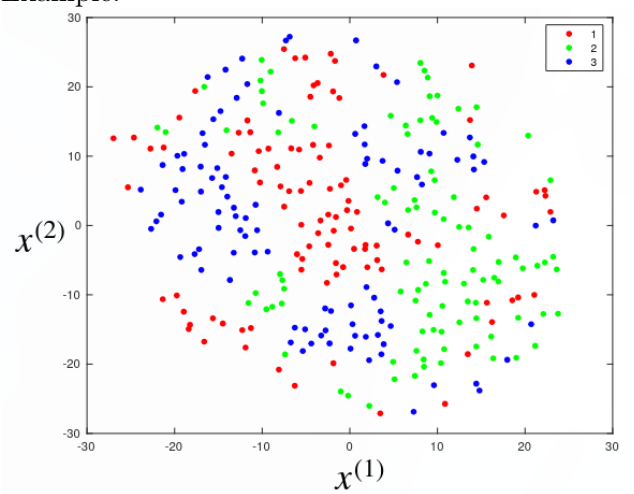
We denote the ith data sample (input) in the collection of N samples as

$$x_i \in \mathbb{R}^D,$$

a column vector of dimension D.
We denote the ith label (output) in the collection of N samples as y_i.
y_i :

- for classification represents the category the sample belongs to.
- for regression, it can be a single continuous value, or a column vector of dim C.

Example:



Here, x_i is one 2D point and thus D = 2, y_i is a discrete value indicating the class (color), i.e, 1, 2 or 3.

2.2 Linear Regression

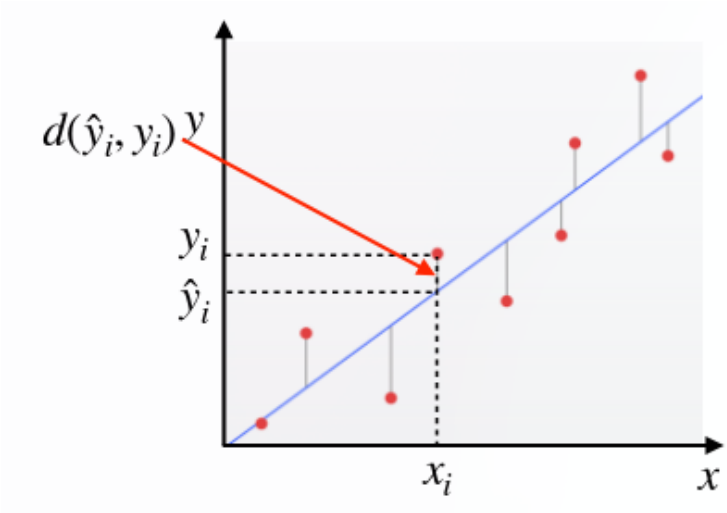
2.2.1 Training

Given N annotated data samples (x_i, y_i) with measurements, we can plot the data, and we will approximate it with a line. Our goal is to find the line that bests fits these measurements →linear regression. It consists of finding the best line parameters $w^{(0)*}$ and $w^{(1)*}$ for some given data.
This corresponds to the training stage:
Given N training pairs (x_i, y_i) , we aim to find $\left(w^{(0)}, w^{(1)}\right)$, s.t the predictions of the model

$$\hat{y}_i = w^{(0)} + w^{(1)} x_i$$

are close to the true values y_i.
The mesure of closeness/error between y_i and $\hat{y}_i$ is represented by the Euclidean distance:

$$d\left(\hat{y}_i, y_i\right) = \sqrt{\left(\hat{y}_i - y_i\right)^2}$$



$\hat{y}_i - y_i$: often reffered to as the *residual*

In practice, one often prefers using the squared Euclidian distance

$$d^2\left(\hat{y}_i, y_i\right) = \left(\hat{y}_i - y_i\right)^2$$

Training can then be expressed as the least-squared minization problem

$$min_{w^{(0)},w^{(1)}} \frac{1}{N} \sum_{i=1}^{N} d^2(\hat{y}_i, y_i)$$

2.2.2 Prediction

Once the best line for the given N observations is found, we can use it to predict a y value for a new x. Let $w^{(0)*}$ and $w^{(1)*}$ be the best line parameters given the observations.
Then

$$\hat{y} = w^{(0)*} + w^{(1)*}x$$

is the estimate value of y for any x.

Lecture 2 — 2026-02-24: Linear Regression (continuous insight)

2.2.3 1D linear regression

With 1D inputs, training a linear regressor aims to minimize

$$R\left(w^{(0)}, w^{(1)}\right) = \frac{1}{N} \sum_{i=1}^{N} d^2\left(\hat{y}_i, y_i\right) = \frac{1}{N} \sum_{i=1}^{N} \left(w^{(0)} + w^{(1)}x_i - y_i\right)^2$$

Equivalently, we can write

$$R\left(w\right) = \frac{1}{N} \sum_{i=1}^{N} \left(w^T x_i - y_i\right)^2 = \frac{1}{N} \sum_{i=1}^{N} \left(x_i^T w - y_i\right)^2$$

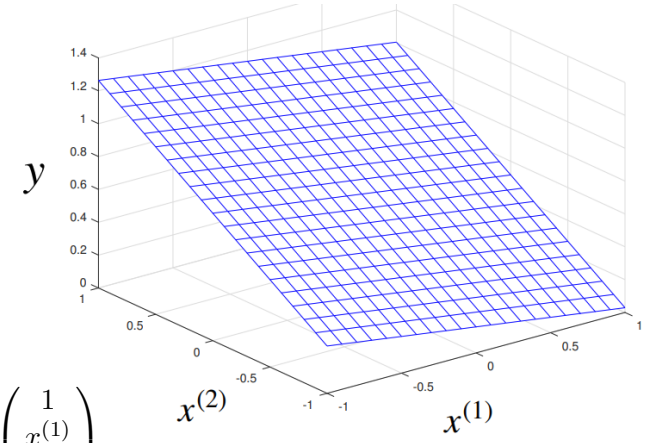
where

$$x_i = \begin{pmatrix} 1 \\ x_i \end{pmatrix} \in \mathbb{R}^2 \qquad w = \begin{pmatrix} w^{(0)} \\ w^{(1)} \end{pmatrix} \in \mathbb{R}^2$$

2.2.4 Two input dimensions

Here instead of a line, we can define a plane.
Expressed

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \end{pmatrix}$$



2.2.5 Hyperplane

This can be generalized to higher dimensions.
In dimension D we can write

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + \ldots + w^{(D)}x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

Whatever the dimension, we can write

$$y = w^T x \quad x \in \mathbb{R}^{D+1}$$

, where the extra dimesion contains a 1 to account for $w^{(0)}$

Because the output remains 1D, we can still use the same least square loss function, and write training as

$$min_w \frac{1}{N} \sum_{i=1}^{N} d^2 \left(\hat{y}_i, y_i\right) \iff min_w \frac{1}{N} \sum_{i=1}^{N} \left(x_i^T w - y_i\right)^2$$

Solving this minimization problem involves computing its derivatives (derivation and gradient) w.r.t the model parameters (w).

2.2.6 Minimizing multivariate function

As in the 1D case (derivative = 0), to minimize a function, we seek the point w^* where the gradient where the gradient vanishes, i.e,

$$\nabla_w R(w^*) = 0$$

2.2.7 Solution of the linear regression

With our least-square formulation, we seek to minimize

$$R(w) = \frac{1}{N} \sum_{i=1}^{N} (x_i^T w - y_i)^2$$

This is a convex function of **w**, which means that we search for parameters w^* s.t

$$\nabla_w R(w^*) = 0$$

Let us start by computing the gradient $\nabla_w R(w)$

Hint 1: $R(w)$ is an average over the N training samples

- As the gradient of an average is the average of the gradients, you can focus on computing the gradient

$$\nabla_w (x_i^T w - y_i)^2$$

Hint 2: Use the chain rule

- The function $(x_i^T w - y_i)^2$ has the form $f(g(w))$, with $f(g) = g^2$
- You can compute its gradient as $\frac{df(g)}{dg} \nabla_w g(w)$

Hint 3: To compute the gradient of g(w), you can make use of the following general vector derivative rule:

- Given 2 vectors (a,b) in $\mathbb{R}^D$

$$\nabla_b (a^T b) = a$$

So with the chain rule:

$$\nabla_w R = \frac{1}{N} \sum_{i=1}^{N} \frac{d(x_i^T w - y_i)^2}{d(x_i^T w - y_i)} \cdot \nabla_w (x_i^T w - y_i) = \frac{2}{N} \sum_{i=1}^{N} (x_i^T w - y_i) \cdot x_i$$

To obtain the solution, we search for w^* s.t

$$\nabla_w R(w^*) = \frac{2}{N} \sum_{i=1}^{N} x_i (x_i^T w^* - y_i) = 0$$

This means

$$(\sum_{i=1}^{N} x_i x_i^T) w^* = \sum_{i=1}^{N} x_i y_i$$

$(D+1) \times (D+1)$ matrix * $(D+1) \times 1$ vector = $(D+1) \times 1$ vector

Let us now group all x_i and all y_i in a matrix and a vector

$$x = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(D)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(D)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_N^{(1)} & x_N^{(2)} & \dots & x_N^{(D)} \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

Then we can re-write the solution as

$$X^T X w^* = X^T y$$

This finally give us

$$w^* = (X^T X)^{-1} X^T y = X^\dagger y$$

where $X^\dagger$ is known as the Moore-Penrose pseudo-inverse of X.

2.2.8 Test time

So now the predicted value $\hat{y}_t$ becomes

$$\hat{y}_t = \begin{pmatrix} 1 \\ x_t \end{pmatrix}^T w^* = (w^*)^T \begin{pmatrix} 1 \\ x_t \end{pmatrix}$$

2.3 Model evaluation

Once an ML model is trained, one would typically understand how well it performs on unseen test data

- At this stage, the parameters of the model are fixed
- Recall that the training and testing data must be separated!

During this evaluation, one compares the predictions of the model with the true annotations of the test data

- In contrast to the training stage, the model parameters are not updated
- The evaluation metric may directly be the loss function, but may also differ from it

2.3.1 Evaluation metrics for regression

Mean Squared Error (MSE): Same as the loss function but for N_t test samples.

$$MSE = \frac{1}{N_t} \sum_{i=1}^{N_t} (\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the prediction for the test sample i and y_i the corresponding ground-truth value

Root Mean Squared Error (RMSE): Square-root of the MSE
Mean Absolute Error (MAE)

$$\frac{1}{N_t} \sum_{i=1}^{N_t} |\hat{y}_i - y_i|$$

Mean Absolute Percentage Error (MAPE)

$$MAPE = \frac{1}{N_t} \sum_{i=1}^{N_t} \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

Example, UCI Wine Quality dataset, predict the quality of wine based on several attributes

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Here the final RMSE is 0.65 for the training data and 0.63 for the test data

One can the look at the coefficient values $w^{(i)}$ to see the influence of each attribute
Warning: The magnitude of a coefficient will depend on the magnitude of the corresponding feature/attribute (A coefficient might be very small simply to compensate for the fact that the range of the feature is very large)

	Coeffecient
fixed acidity	0.017737
volatile acidity	-0.992560
citric acid	-0.139629
chlorides	-1.590943
free sulfur dioxide	0.005597
total sulfur dioxide	-0.003520
density	0.768590
pH	-0.437414
sulphates	0.812888
alcohol	0.301484

2.3.2 Linear regression with multiple output dimensions

Example of multiple output: human pose estimation

2.3.3 Model

The linear model has to be modified to output multiple values (not a vector w anymore because $w^T x_i$ output a single value)
So we use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$ s.t

$$\hat{y}_i = W^T x_i = \begin{pmatrix} w_{(1)}^T \\ w_{(2)}^T \\ \vdots \\ w_{(C)}^T \end{pmatrix} x_i$$

where each $w_{(j)}$ is a $(D+1)$-dimensional vector, used to predict one output dimension.
The multi-output linear regression is written

$$min_w \sum_{i=1}^{N} \left\| W^T x_i - y_i \right\|^2$$

where $\|a\| = \sqrt{\sum_{i=1}^{C} (a^{(j)})^2}$ indicates the norm of a vector (here a $\in \mathbb{R}^C$

To compute the gradient of this function, we make use of the following properties:
1. Norm and derivative:

$$\|a\|^2 = a^T a$$

and thus

$$\nabla_a \|a\|^2 = 2a$$

2. Matrix derivative:

$$\nabla_B (a^T B) = a$$

Thus using the chain rule we obtain the gradient

$$\nabla_W R = 2 \sum_{i=1}^{N} x_i (x_i^T W - y_i^T)$$

setting it to zero means that

$$(\sum_{i=1}^{N} x_i x_i^T) W^* = \sum_{i=1}^{N} x_i y_i^T$$

$(D+1) \times (D+1)$ matrix * $(D+1) \times C$ matrix = $(D+1) \times C$ matrix
We can again group the inputs in a matrix $\mathbf{X} \in \mathbb{R}^{N\times(D+1)}$, but the outputs now need to be grouped in a matrix (not a vector anymore), as

$$Y = \begin{pmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_N^T \end{pmatrix} = \begin{pmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(C)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(C)} \\ \vdots & \vdots & \vdots & \vdots \\ y_N^{(1)} & y_N^{(2)} & \cdots & y_N^{(C)} \end{pmatrix} \in \mathbb{R}^{N\times C}$$

Then, following the same strategy as before, we have

$$W^* = (X^T X)^{-1} X^T Y = X^\dagger Y$$

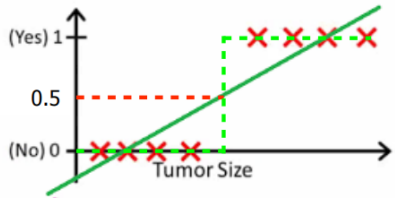
2.4 Linear classification

To predict one discrete label for a given sample e.g., binary classification or movies (like vs not), multi-class image recognition.
Classification can be tackle as a regression task (given a threshold we can classify the output in categories).

E.g. Classify a tumor as benign vs malignant

- 1D input (tumor size), 1D output (malignant/benign)
- Using a linear model, we can predict the label $\hat{y} = w^{(0)} + w^{(1)} x$

$$\text{label} = \begin{cases} 1 & \text{if } \hat{y}_t \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \qquad \text{with } \hat{y}_t = \mathbf{x}_t^T \mathbf{w}^*$$



To fit the linear model to training data, the same strategy as for linear regression can be used:

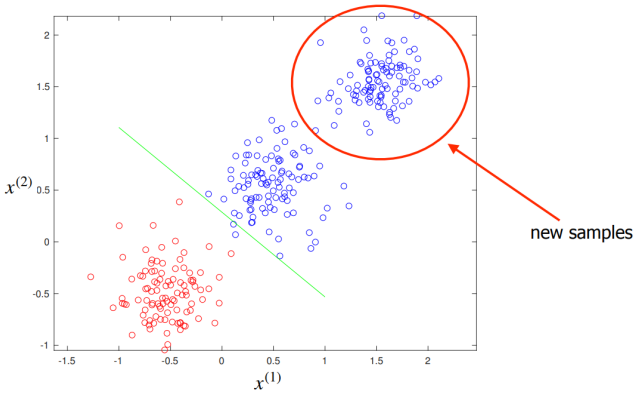
- Given N pairs (x_i, y_i) , we can use the least-square loss to write

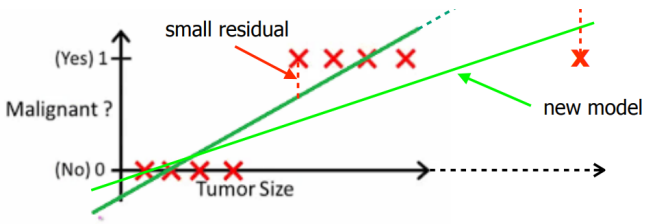
$$min_w \sum_{i=1}^{N} (x_i^T w - y_i)^2$$

So because the formulation is the same, the solution too

$$w^* = (X^T X)^{-1} X^T y$$

However, this model isn't that good because with some data it the **decision boundary**(threshold) will not react well and misclassified some samples.





So we will introduce an other model the **logistic regression**.

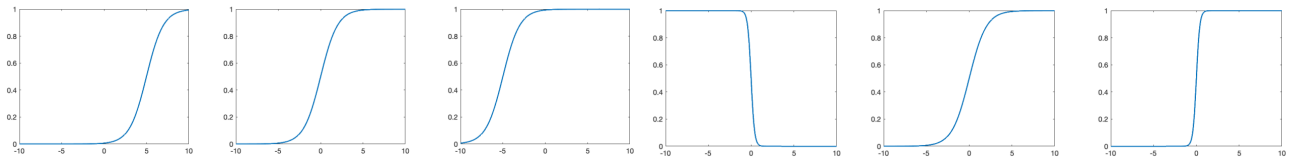
Definition 3 (Loss function)**.** The loss function (general ML concept) : $l(\hat{y}_i, yi)$ computes an error value between the prediction and the true value.

2.4.1 Logistic regression

The idea of the logistic regression is that first we compute $y = w^T x$ and then we apply the sigmoid on y written $\sigma(y)$, to establish a probability $p(y = 1|x)$. Then the decision boundary is obtained when the probability equals (0.5) .
Logistic function :f(a) $= \frac{1}{1+exp(-a)}$
Applying it to the output of a linear model gives a prediction:

$$\hat{y} = \frac{1}{1 + exp(-w^{(0)} - w^{(1)}x)}$$

This is a smooth approximation of the step function (avoid the gradient that is zero almost everywhere).



$w^{(0)} = -5, w^{(1)} = 1$ $w^{(0)} = 0, w^{(1)} = 1$ $w^{(0)} = 5, w^{(1)} = 1$ $w^{(0)} = 0, w^{(1)} = -5$ $w^{(0)} = 0, w^{(1)} = 1$ $w^{(0)} = 0, w^{(1)} = 5$

Then the data fit much better even with the presence of outliers

In the binary case, with D dimensional inputs, we can write the prediction of the model (logistic regression) as

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + exp(-w^T x)}$$

it can be interpreted as the probability that **x** belongs to the positive class $\rightarrow 1 - \hat{y} =$ proba. negative class.
Notes, in the logistic regression:

$$y \in \{0, 1\} \text{ with } 0 \rightarrow \text{negative class, } 1 \rightarrow \text{positive class}$$

$$\text{The model computes: } \hat{y} = P\left(y = 1|x\right)$$

Even that the name of this model is logistic **regression**, this is truly a classification model, not a regression one

Training

To learn the parameters **w** need a loss function:
Cross-entropy :

1D case

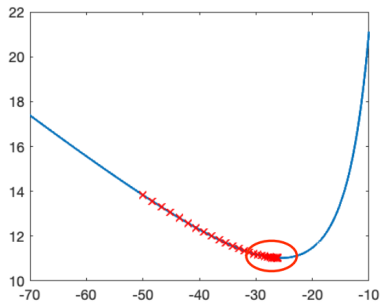
$$R(w) = -\sum_{i=1}^{N}(y_i ln(\hat{y}_i) + (1 - y_i)ln(1 - \hat{y}_i)) = -\sum_{i \in positivesamples} ln(\hat{y}_i) - \sum_{i \in negativesamples} ln(1 - \hat{y}_i)$$

$\rightarrow$as y_i is either 0 or 1
It's a **convex** function, however, its minimization has no closed-form solution. So we optimize it via

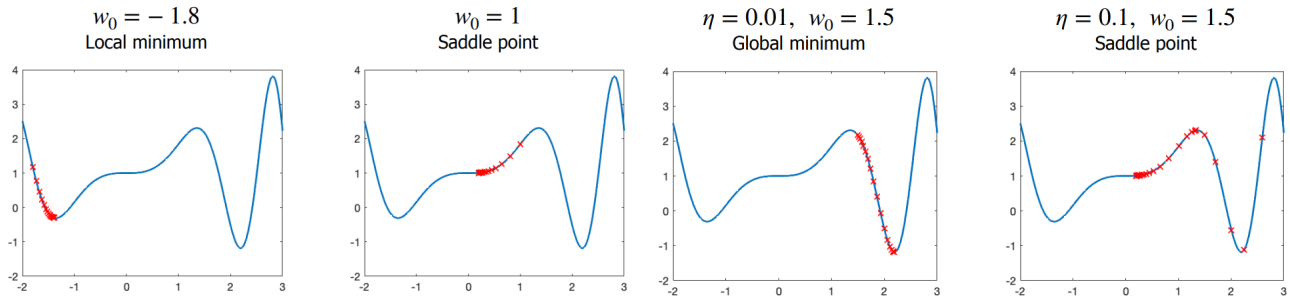
gradient descent.
The **algorithm** for minimization is:

- 1. Initialize w_0 (e.g, randomly)
- 2. While not converged $\implies$ *update* $\mathrm{w}_k \leftarrow w_{k-1} - \eta \frac{dR(w_{k-1})}{dw}$ with η the step size of each iteration (learning rate)

This algorithm converges because the derivative becomes smaller ($\frac{dR(w)}{dw} \to 0$ (∇w becomes smaller).
The η influences the speed of the convergences (doing bigger or smaller steps), but having too large steps may make the algorithm to jumps between two points and never converge.
As the initial w_0 is the starting point, having a w_0 near the minimum will makes the algorithm converges faster.



To minimize **non-convex** function the same algorithm can be used. However, different initizialisations yield to different results because there is more than one point with derivative equal to zero.



D dimensions case As in practice we have more than one parameter (i.e $\mathbf{w} \in \mathbb{R}^{D+1}$), we use the function gradient:

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w^{(0)}} \\ \frac{\partial R}{\partial w^{(1)}} \\ \vdots \\ \frac{\partial R}{\partial w^{(D)}} \end{pmatrix} \in \mathbb{R}^D$$

Recap : the gradient at a point w has the direction of the greatest increase of the function at w and its magnitude is the rate of increase in that direction.
So to decrease the function we therefore move in the direction opposite to the gradient.
Gradient descent is used to find the *argmin* of a function, i.e., we want to find the values of parameters x such that a function f is minimized :

$$x^* = \underset{x}{\operatorname{argmin}}\, f(x)$$

Algorithm - gradient descent

- 1. Initialize w_0 (e.g, randomly)
- 2. While not converged $\implies$ *update* $\mathrm{w}_k \leftarrow w_{k-1} - \eta \nabla_w R(w_{k-1})$

Cross-Entropy (multiple outputs):

$$R(w) = -\sum_{i=1}^{N} y_i ln\sigma(w^T x_i) + (1 - y_i) ln(1 - \sigma(w^T x_i))$$

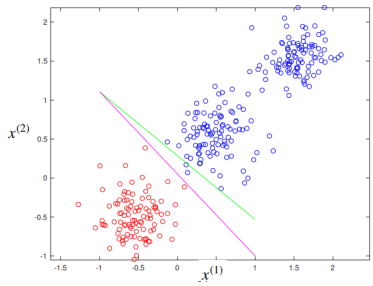
$$= -\sum_{i \in positivessamples} ln\sigma(w^T x_i) - \sum_{i \in negativesamples} ln(1 - \sigma(w^T x_i))$$

To minimize it via gradient descent we compute its gradient w.r.t **w**

$$\nabla_w R(w) = \sum_{i=1}^{N} (\hat{y}_i - y_i) x_i$$

where y_i is the true label for sample i, i.e., 1 for positive samples and 0 for negative samples.

And now, even with outliers, the boundary remains correct (Least-square classification in green, logistic regresion in magenta)



2.4.2 Difference between linear models

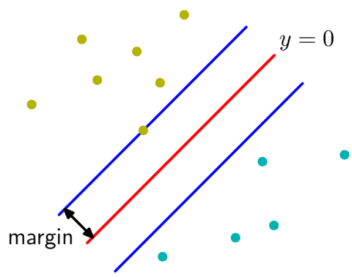
The real difference between linear models is the **loss function**, it has an impact on the resulting algorithm.

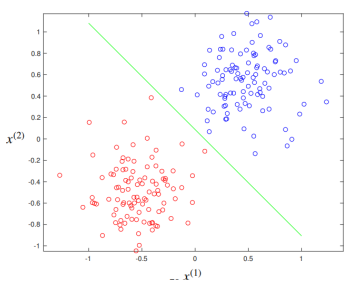
2.4.3 Support Vector Machine (SVM)

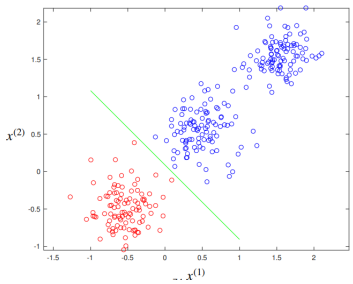
The SVM is a binary classifier based on the linear model as before:

$$\hat{y} = w^T x$$

It aims to maximize the margin (the distance between the decision boundary and the nearest sample) as a good decision boundary should maximize it, while enforcing a correct classification for each training samples.







So here we see that adding a new samples does not change the decision boundary as the margin stay the same.

2.4.4 Classification evaluation

For binatry case we could use confusion matrix:
TP: True positive (number of samples correctly classified as positive)
FP: False positive (number of samples incorrectly classified as positive)
TN: True negative (number of samples correctly classified as negative)
FN: False negative (number of samples incorrectly classified as negative)
P: Total number of positive samples
N: Total number of negative samples

True class → Hypothesized class ↓	Pos	Neg
Yes	TP	FP
No	FN	TN
	P=TP+FN	N=FP+TN

Definition 4 (Accuracy)**.** Percentage of correctly classified samples

$$Acc = \frac{TP + TN}{P + N}$$

However, the accuracy doesn't give the information about the behavior of a model, in fact models with same accuracy could have a totally different behavior(e.g, one performing poorly on positive recogniton and well on negative one and the other one the total contrary).
Thus we could use other metrics in complement:

Definition 5 (Precision)**.** Percentage of samples classified as positives that are truly positives

$$Precision = \frac{TP}{TP + FP}$$

Definition 6 (Recall)**.** Percentage of positive samples that are correctly classified as positives

$$Recall = \frac{TP}{P}$$

But, still two classifiers with the same precision and recall could have a very different behavior as before (checking for accuracy could clearly show this), so we could too look for :

Definition 7 (False positive rate)**.** Percentage of negative samples that are incorrectly classified as positives

$$FP\ rate = \frac{FP}{N}$$

Definition 8 (F1 score)**.** Single number that combines precision and recall

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

ROC curves

Many classifiers output a score/confidence for their predictions (e.g., $\hat{y}$ is a continuous value between 0 and 1 in logistic regression).
The decision between positive and negative can be computed by thresholding this score:

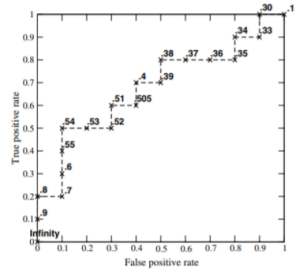
$$\text{label}_i = 1 \text{ if } \hat{y}_i \geq \tau$$

where τ is a given threshold

The Receiver Operating Characteristic (ROC) curve plots the true positive rate (recall) as a function of the false positive rate, obtained by varying the score threshold.

Inst#	Class	Score	Inst#	Class	Score
1	p	.9	11	p	.4
2	p	.8	12	n	.39
3	n	.7	13	p	.38
4	p	.6	14	n	.37
5	p	.55	15	n	.36
6	p	.54	16	n	.35
7	n	.53	17	p	.34
8	n	.52	18	n	.33
9	p	.51	19	p	.30
10	n	.505	20	n	.1

Samples sorted by score
(*Class* denotes the true label)



ROC curve

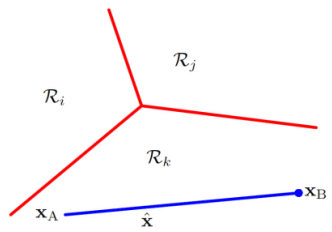
This model give a score(probability) to each observation(here Inst) and make the threshold vary, then compute the TP and FP rate, and plot the curves.
A good model as a plot near the top left corner (meaning that the TP rate is high and the FP rate low) as we mesure the quality of the model with the Area under the curve **(AUC)**(goes from 0 to 1), 1.0 that the model is perfect, 0.5 is the result if the model makes random predictions, and less is worse than randomness.

2.4.5 From binary to multi-class classification

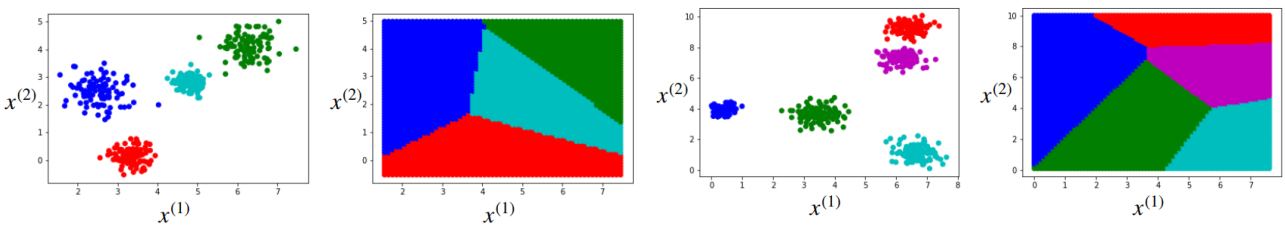
What if we have more than two classes? How our three linear models can handle this scenario ?

2.4.6 Decision boundaries: General Definition

First let's update our knowledge on decision boundaries, as here we are not anymore treating binary label.
The decision boundary between two classes k and j is defined as the set of points
$\mathbf{x}$ such that $\hat{y}^{(k)}(x) = \hat{y}^{(j)}(x)$.
This corresponds to the $(D-1)-$dimensional hyperplane.
The decision regions are **singly-connected and convex** (any point $\hat{x}$ between two points x_A and x_B in the same region (e.g, k) will also be in the same region)



Decision boundaries for C = 4 and C = 5 classes.



2.4.7 Classification evaluation for Multi-class

Confusion matrix for multi-class case.

		Predicted		
		Iris-setosa	*Iris-versicolor*	*Iris-virginica*
Actual	*Iris-setosa*	12	1	1
	Iris-versicolor	0	16	0
	Iris-virginica	0	1	16

Accuracy : A global accuracy can be obtained by summing the correct predictions for all classes (diagonal values of the confusion matrix), and dividing this by the total number of samples (sum of all entries in the confusion matrix)
Precision, recall, FP-rate and F1-score: The simplest approach to handling these metrics is to compute them for each individual class, and then average over the classes
In this case, each individual class is in turn considered as the "positive" class while the others act as the "negative" class
This does not account for class imbalance (some classes may have many more samples than others; more about this in a future lecture)
This can be handled by weighting the metric for each class by a value depending on the proportion of samples from that class

2.4.8 Least-square classification

The most common approach to modeling a multi-class problem consists of encoding the class label as a vector with a single 1 at the index corresponding to the category and 0s elsewhere.
e.g. in a 5-class problem, a sampe in class 2 is represented as:

$$y = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

This is referred as **one-hot-encoding**.
As this is similar to making the model output multiple (as for multi-output regression), we therefore again use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$, s.t

$$\hat{y_i} = W^T x_i = \begin{pmatrix} w_{(1)^T} \\ w_{(2)^T} \\ \vdots \\ w_{(C)^T} \end{pmatrix} x_i$$

where each $w_{(j)}$ is a $(D+1)-$dimensional vector, used to predict one output dimension, i.e., the **score** for one class.

Training

We can then write training as

$$min_w \sum_{i=1}^{N} \left\| W^T x_i - y_i \right\|^2$$

where each y_i is now a $C-$dimensional vector with a single one at the inex corresponding to the class label and zeroes everywhere else.

Solution

The inputs and the outputs are two matrices:

$$X = \begin{pmatrix} x_i^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} \in \mathbb{R}^{N\times(D+1)} \quad Y = \begin{pmatrix} y_i^T \\ y_2^T \\ \vdots \\ y_N^T \end{pmatrix} \in \mathbb{R}^{N\times C}$$

X = data $\quad Y$ = True classes
N = number of samples $\quad D$ = number of features (+1 for bias) $\quad C$ = number of classes
Then, the solution has the same form as in the regression case (obtained by zeroing out the gradient w.r.t. W), i.e.,

$$W^* = (X^T X)^{-1} X^T Y = X^\dagger Y$$

W = weight matrix (i.e $W = [W_0, W_1]$, $\quad W_0, W_1$ weight classes 0 and 1)

Prediction

Given the optimal parameters, the prediction for a new sample is computed as

$$\hat{y} = (W^*)^T x$$

The sample is then assigned to class k if

$$\hat{y}^{(k)} > \hat{y}^{(j)}, \quad \forall j \neq k$$

In other words the final label k is obtained as

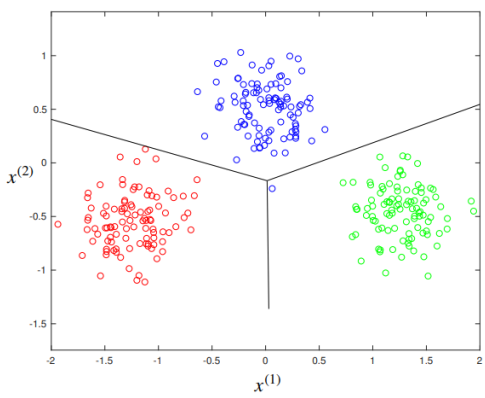
$$k = \underset{j}{\operatorname{argmax}}\, \hat{y}^{(j)}$$

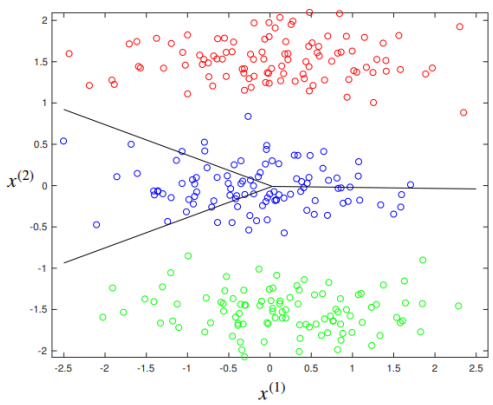
That is, the label is the index (position) of the maximum value in the vector $\hat{y}$
I.e: if we have

$$\hat{y} = \begin{pmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \\ \hat{y}^{(3)} \end{pmatrix} = \begin{pmatrix} 2.3 \\ 0.7 \\ -1.5 \end{pmatrix}$$

here the argmax_j(biggest score) is 2.3, so the label predicted is the label 1.

(boundary where $\hat{y}_i = \hat{y}_k$ (for 2 different classes)) Example of success and failure case of 2D inputs and 3 classes least-square classification:





Drawbacks

As in the binary case, the multi-class least-square classification suffer from outliers, because it tries to minimize the distance (regression). So we need an other model like the logistic regression one but for multi-class: **Multi-class logistic regression** :O !!

2.4.9 Multi-class logistic regression

We still use the one-hot enconding to represent multi-class labels.
We again need to use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$ to represent the model parameters.
But now, the probability for a class k is given by the **softmax** function (takes scores and transform them into probabilities)

$$\hat{y}^{(k)}(x) = \frac{\exp(w_{(k)}^T x)}{\sum_{j=1}^{C} \exp(w_{(j)}^T x)}$$

The empirical risk can then be derived from the multi-class version of the cross entropy, which yields

$$R(W) = -\sum_{i=1}^{N} \sum_{k=1}^{C} y_i^{(k)} \ln \hat{y}_i^{(k)}$$

(i →go through the samples, k →go through the classes)
As in the binary case :

- a single $y_i^{(k)}$ is 1 for every sample i, so we really have one term per training sample. (as if sample k belongs to class i then only the term for this class/label stay in the sum (other are 0))
- training is done by minimizing this empirical risk using gradient descent

Gradient

The gradient descent is the same algorithm as for the muti-outputs binary case, the only difference is that the parameters are shaped as a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$, not a vector anymore :
Algorithm - gradient descent

- 1. Initialize W_0 (e.g, randomly)
- 2. While not converged $\implies$ *update* $\mathrm{W}_k \leftarrow W_{k-1} - \eta \nabla_w R(W_{k-1})$

Thus the gradient will also be a matrix

$$\nabla_W R(W_{k-1}) = \sum_{i=1}^{N} x_i (\hat{y}_i - y_i)^T \quad \in \mathbb{R}^{(D+1)\times C}$$

Prediction

At test time, given a new input sample $\mathbf{x}$, the probability/score for any class k is computed via the softmax function as

$$\hat{y}^{(k)}(x) = \frac{\exp((w_{(k)}^*)^T x)}{\sum_{j=1}^{C} exp((w_{(j)}^*)^T x)}$$

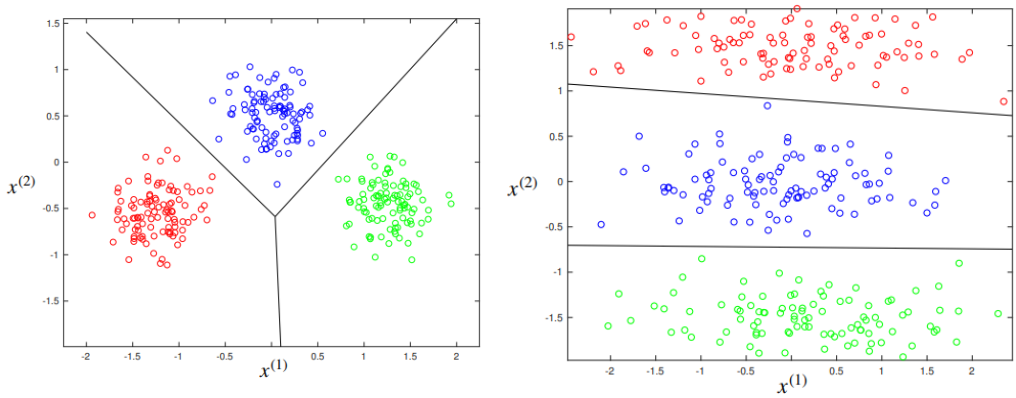
The final class label is then predicted as

$$k = \underset{j}{\operatorname{argmax}}\, \hat{y}^{(j)}(x)$$

That is, as in least-square classification, the label is the index (position) of the maximum value in the vector $\hat{y}$

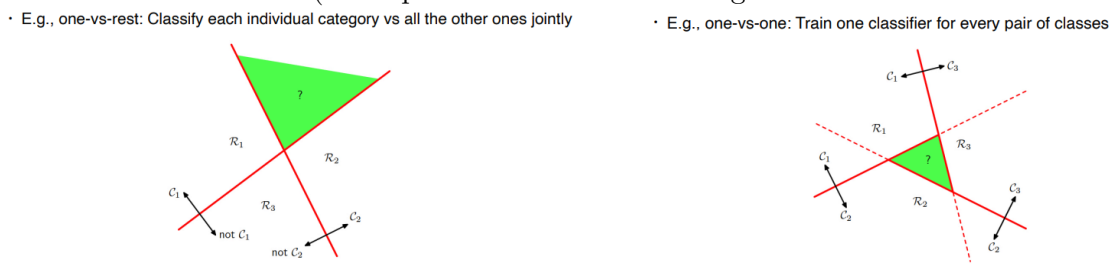
Example

2D inputs, 3 classes: Logistic regresison results



2.4.10 Dealing with multiple classes

SVM cannot be extended to the multi-class scenario.
Instead one can use several binary classifier E.g, one-vs-rest or one-vs-one (however leaves ambiguities and inconsistencies (a sample can be classified being from more that one class at a time))

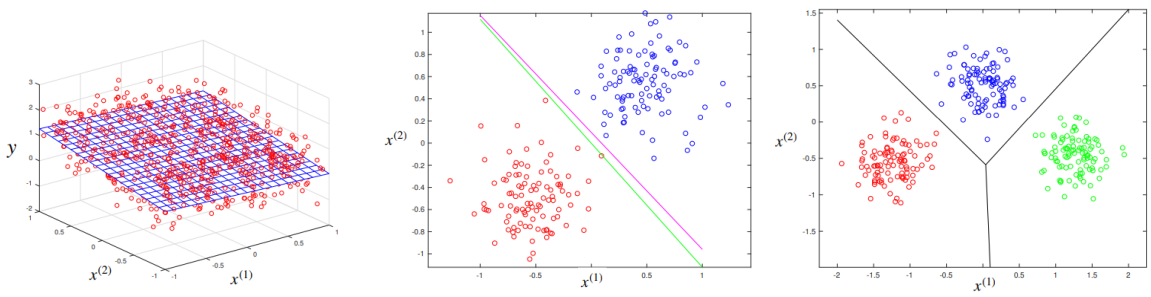


2.4.11 Comparing linear classifiers

We have now seen 3 different linear(multi-class) classifiers: Least-square classification, Logistic regression, SVM.
In practice, one almost never uses the least-square one (only for pedagogiqual standpoint).
Depending on the data to classify, we may use Logistic regression or SVM as they are both better than the other one depending of the data.

2.4.12 Recap - and new challenge

Linear regression : Fit a linear model to the observations
Binary linear classification: Use a linear model to separate the positive and negative samples
Multi-class linear classification: Use a linear model to separate the classes, via a one-hot encoding of the label



Unfortunately, no linear model can directly fit non linear data (we will go from linear to non linear model)